

Текст для презентации

«Сравнение эффективности Q-Learning и DQN на случайных и детерминированных картах»

Слайд 1 — Титульный

Добрый день. Тема моей работы — сравнение эффективности алгоритмов Q-Learning и DQN на случайных и детерминированных картах. Для исследования была разработана полноценная платформа: игровой движок с процедурной генерацией подземелий и обучаемый нейросетевой агент.

Слайд 2 — Актуальность и цель

Интерес к данной теме обоснован историческими прорывами в области глубокого обучения с подкреплением. В 2013 году DeepMind опубликовала алгоритм DQN, который превзошёл человека в 49 из 57 игр Atari, используя только пиксели экрана. В 2016 году появился Double DQN, устраняющий систематическое завышение Q-значений. Наш проект реализует аналогичные методы на задаче прохождения процедурно-генерируемых подземелий через 6 фаз курикулума — от простой навигации до финальной карты 32×32 с врагами.

Слайд 3 — Reinforcement Learning

Обучение с подкреплением — это парадигма машинного обучения, в которой агент обучается методом проб и ошибок, взаимодействуя со средой. Ключевых элементов три: агент принимает решения и совершает действия, среда реагирует на эти действия, а награда — это обратная связь, которая говорит агенту, насколько хорошо он справился. Цель агента — максимизировать суммарную накопленную награду за весь эпизод.

Слайд 4 — Архитектура системы

Система состоит из двух независимых компонентов, взаимодействующих через протокол stdin/stdout. Игровой движок написан на C# с использованием .NET 8 и SFML — он отвечает за генерацию карт, симуляцию врагов и расчёт наград. Python-агент реализует нейросеть на PyTorch: вектор наблюдений из 1157 признаков проходит через два скрытых слоя и возвращает 7 Q-значений. Курикулумный менеджер управляет прогрессией сложности, а детектор стагнации предотвращает зависание обучения.

Слайд 5 — Игровой движок

Карты генерируются алгоритмом BSP — бинарного разбиения пространства. Один и тот же seed всегда даёт одну и ту же карту, что обеспечивает воспроизводимость экспериментов. Игрок всегда появляется в точке, максимально удалённой от выхода по BFS. Враги трёх типов преследуют игрока только при прямой видимости — используется алгоритм Брезенхема. Скорость врагов дробная: они двигаются через накопление энергии, что создаёт асинхронное поведение.

Слайд 6 — Пространство наблюдений и действий

Каждое наблюдение — вектор из 1157 чисел. Основная часть — локальный вид 33×33 клетки вокруг игрока, то есть 1089 значений. Глобальная миникарта 8×8 строится через max-pooling — это гарантирует, что выход с кодом 8 (наивысший в иерархии) всегда виден на миникарте, даже на карте 64×64 . Четыре скаляра кодируют HP, прогресс эпизода и относительное направление к выходу. Агент располагает 6 действиями: движение в 4 стороны, удар дугой и выстрел лучом.

Слайд 7 — Алгоритмы

В проекте реализованы три алгоритма. Q-Learning — базовый: без буфера и целевой сети, обновление на каждом шаге. Из-за отсутствия буфера

воспроизведения он страдает от катастрофического забывания. Vanilla DQN добавляет буфер на 200 тысяч переходов и целевую сеть — это стабилизирует обучение. Double DQN идёт дальше: он разделяет выбор действия и оценку его ценности между онлайн- и целевой сетями, что устраняет систематическое завышение Q-значений, характерное для DQN.

Слайд 8 — Система наград

Система наград построена вокруг двух принципов. Первый — нормализация: штраф за урон считается как процент от максимального HP, поэтому сигнал одинаков во всех фазах курикулума независимо от того, 15 у агента HP или 75. Второй — BFS-шейпинг: приближение на одну клетку к выходу даёт +0,5, удаление — -0,5. Ключевое свойство — сумма наград по любому замкнутому маршруту равна нулю, что принципиально исключает накрутку очков.

Слайд 9 — Детектор стагнации

В ходе обучения агент часто попадал в локальный оптимум: win_rate переставал расти задолго до порога перехода в следующую фазу. Решением стал детектор стагнации: каждые 300 эпизодов сравниваются последние 500 эпизодов с предшествующими 500. Если win_rate упал более чем на 2%, epsilon умножается на 2,0 — это вынуждает агента снова исследовать среду. Между срабатываниями выдерживается кулдаун в 1800 эпизодов, чтобы избежать дестабилизации.

Слайд 10 — Поэтапное обучение

Курикулум состоит из 6 фаз. Фаза 0 — навигация без врагов на карте 16×16 — учит агента находить выход в принципе. Далее постепенно добавляются враги, растут карта и HP игрока. Для перехода нужно набрать пороговый win_rate — от 50% в фазе 0 до 25% в фазе 5. При каждом переходе буфер воспроизведения очищается от устаревших данных, epsilon

сбрасывается вверх для переисследования новой карты, и win_rate начинает считаться заново.

Слайд 11 — Серия экспериментов

Работа прошла через пять итераций. В начале агент вообще не мог обучаться: критический баг в подсчёте побед, несбалансированные награды. После добавления курикулума и детектора стагнации обучение пошло, но выявилась дестабилизация при слишком агрессивном bump epsilon — коэффициент снижен. Анализ показал, что пороги фаз были нереалистично высокими — изменены на нисходящий ряд 50/45/40/35/30/25%. Финальное сравнение трёх алгоритмов дало однозначный результат: DDQN прошёл все 6 фаз за 12 512 эпизодов.

Слайд 12 — Результаты

DDQN продемонстрировал наилучший результат: все 6 фаз за 12 512 эпизодов. Q-Learning застрял уже в фазе 1 и так и не смог пройти навигацию без врагов. Интересна двухстадийная динамика на финальной карте 64×64: при высоком epsilon агент гибнет в 93% эпизодов — он ещё не умеет выживать. После снижения epsilon смерти падают до 29%, но таймауты вырастают до 80% — агент научился выживать, но теряется при навигации на большой карте.

Слайд 13 — Практическое применение

DDQN находит применение далеко за пределами игр. В робототехнике он управляет манипуляторами и маршрутами складских роботов. В сетях — оптимизирует маршрутизацию пакетов; DeepMind применила схожий подход в дата-центрах Google, снизив энергопотребление на охлаждение на 40%. В финансах агент принимает торговые решения, опираясь на исторические данные. В медицине — оптимизирует схемы лечения и радиационную

терапию. В кибербезопасности — автоматизирует тестирование на проникновение.

Слайд 14 — Выводы

Подведём итоги. Первое: Q-Learning категорически непригоден для задач с нестабильной средой — без буфера воспроизведения катастрофическое забывание не даёт ему обучиться даже базовой навигации. DDQN показал наилучший результат. Второе: промежуточные фазы курикула критически важны — добавление фазы 18×18 подняло win_rate с 4,4% до 15,3% на следующей фазе. Третье: медленный EPS_DECAY=0,9998 продлил период активного исследования второе, предотвратив преждевременную конвергенцию на субоптимальную политику. Спасибо за внимание.